



DOUGLAS COLLEGE

CSIS 4495 – 002 | Applied Research Project

Title: Bookstagram

"Why just read when you can create!"

(MERN Stack Social AI Authoring Platform)

Midterm Report

Midterm Video Demo Link:

<https://drive.google.com/file/d/1QJYPDAUgMnmrUgd-ZJ5ej-NqN9W0BEr/view?usp=sharing>

Submitted To:

Prof. Padmapriya Arasanipalai Kandhadai

Team Members

Name	Student ID	Position
Khushi Bhatt	300394398	Team Leader
Aditi, Aditi	300396361	Team Member

1. Introduction

1.1 Background and Context of the Research

The rapid development of digital platforms and generative artificial intelligence has significantly transformed how content is created and consumed. Traditional book writing has long been considered a time-intensive activity requiring sustained focus and discipline. However, modern digital culture, particularly influenced by platforms such as Instagram and TikTok, has shifted user preferences toward short, interactive, and visually engaging formats. This shift has contributed to reduced attention spans and lower completion rates in long-form writing.

At the same time, generative AI tools such as Google Gemini have made content creation more accessible by assisting users in generating story ideas, outlines, and draft chapters, thereby reducing creative barriers such as writer's block. While these AI tools improve productivity, they typically function as standalone tools and do not incorporate structured social engagement features.

Social media platforms, conversely, encourage interaction through likes, follows, and content feeds but lack tools specifically designed for structured storytelling and long-form writing development. This separation between AI-assisted productivity and socially engaging platforms creates a noticeable gap in the digital publishing ecosystem. The Bookstagram project is positioned at this intersection, combining AI-assisted story generation with interactive social features to provide a unified "social-authoring" experience.

1.2 Problem Statement

Many beginner and intermediate writers struggle to complete long-form writing projects. Common challenges include difficulty generating ideas, lack of motivation, and feeling overwhelmed by the structure of a full eBook, resulting in a high rate of unfinished creative projects.

Although AI tools assist in generating content efficiently, they do not necessarily improve long-term engagement or motivation. Social media platforms promote frequent interaction but do not provide structured support for organized storytelling. This creates a "social-authoring gap," where no unified platform effectively integrates AI-assisted writing with community-based engagement features. Bookstagram directly addresses this gap.

1.3 Research Questions

To guide the development and evaluation of Bookstagram, this research addresses the following questions:

1. How does AI-assisted story generation affect writing productivity and user confidence among beginner writers?

The integration of Google Gemini within the platform is intended to reduce the initial difficulty of starting a story. Early implementation indicates that users can generate outlines and draft chapters more efficiently than manual writing, suggesting a positive influence on writing confidence and productivity.

2. Do social engagement features such as likes, follows, and story feeds improve user motivation and platform retention?

The introduction of social features transforms writing from a private task into an interactive experience. Preliminary observations suggest that users are more likely to revisit the platform when they receive

likes or gain followers, indicating that these mechanisms may contribute to sustained motivation and participation.

3. Can the combination of AI-assisted writing and social networking features provide a more effective digital authoring experience compared to traditional writing tools?

Traditional writing applications primarily focus on editing and formatting, offering limited creative guidance or community interaction. By combining AI generation with social validation, Bookstagram introduces a hybrid model that addresses both productivity and motivational challenges simultaneously.

1.4 Literature Review and Knowledge Gaps

Research shows that generative AI is increasingly used in content creation and digital publishing. AI writing assistants help users overcome creative barriers and increase drafting speed. Studies on human-AI collaboration suggest that users often experience improved productivity when AI functions as a supportive partner rather than a replacement (Bender et al., 2021).

Additionally, research on digital engagement and gamification demonstrates that validation mechanisms such as likes, feedback, and progress tracking enhance user motivation and retention. Social platforms effectively maintain engagement through interactive content ecosystems.

However, most AI writing platforms operate independently of social networking features, while social media platforms lack structured tools for organized storytelling. Limited research has explored how combining AI-assisted writing with social engagement mechanisms may influence writing productivity, completion rates, and user satisfaction. This project aims to address this gap directly.

1.5 Initial Hypotheses and Assumptions

Based on prior research and existing market trends, this study operates under the following hypotheses and assumptions:

Hypotheses:

- AI-assisted story generation will increase writing productivity and reduce writer's block among users.
- Social engagement features will positively influence user motivation and platform retention.
- A combined AI and social-authoring model will provide a more engaging and effective writing experience than traditional standalone writing tools.

Assumptions:

- Users are willing to use AI as a collaborative writing assistant rather than a replacement.
- Younger users prefer interactive and visually structured writing environments over traditional word processors.
- Social validation mechanisms encourage consistent participation in creative tasks.

1.6 Potential Benefits of the Research

The development of the Bookstagram platform has the potential to generate several key benefits:

1. **Improved Writing Productivity**

The integration of AI-assisted writing tools may help users generate structured content more efficiently, reducing writer's block and increasing story completion rates.

2. **Increased Writing Confidence**

Beginner and intermediate writers may feel more confident when supported by AI-generated suggestions and structured outlines.

3. **Enhanced User Motivation**

Social engagement features such as likes, follows, and interactive story feeds may encourage consistent participation and sustained writing activity.

4. **Community Building and Collaboration**

The platform can foster a supportive digital community where writers share ideas, receive feedback, and grow together.

5. **Advancement in Human & AI Collaboration**

The project contributes to research by exploring how AI can function as a collaborative partner rather than a replacement in creative tasks.

6. **Innovation in Digital Publishing Platforms**

By combining AI-assisted content generation with social networking elements, the platform introduces a new "social-authoring" model within digital publishing environments.

2. Summary of the Initially Proposed Research Project

2.1 Research Objectives and Overall Goal

The Bookstagram project was initially proposed as a Social-AI integrated digital storytelling platform designed to address challenges faced by beginner and intermediate writers. The primary objective was to develop a functional prototype that combines AI-assisted content generation with social networking features to improve writing productivity and user engagement.

The project aims to reduce common barriers such as writer's block, lack of motivation, and incomplete writing projects. By integrating Google Gemini AI for story generation and implementing social validation mechanisms such as likes, follows, and story feeds, the platform seeks to transform writing into an interactive and motivating experience.

The specific objectives of the proposed research include:

1. Designing and implementing a MERN-based web application for digital storytelling.
2. Integrating AI-powered story outline and chapter generation using Google Gemini.
3. Developing social interaction features to encourage user engagement.
4. Evaluating whether combining AI productivity tools with social feedback mechanisms increases writing completion rates and user confidence.
5. Collecting and analyzing user interaction data to assess system effectiveness.

2.2 Research Design and Methodology

The proposed study follows an experimental and developmental research design. The primary focus is to build a working prototype and observe how users interact with AI-assisted and socially enhanced writing tools.

An Agile Software Development Life Cycle (SDLC) methodology was adopted to allow incremental development, continuous testing, and refinement of system features. The project was structured into weekly sprints to systematically implement backend infrastructure, frontend interface, AI integration, and social engagement features.

Data collection was planned through two primary methods:

- Automated system analytics stored in MongoDB, including the number of stories created, chapters completed, AI usage frequency, likes received, and follower growth.
- Pre-test and post-test surveys to measure changes in writing confidence, motivation, perceived usefulness of AI assistance, and overall usability.

The target participant group was defined as 15–20 users between the ages of 12 and 25 with beginner to intermediate writing experience.

2.3 Technology Stack and System Architecture

The project was designed using the MERN stack due to its flexibility and unified JavaScript ecosystem.

- **Frontend:** React.js with Tailwind CSS for responsive and modern UI design
- **Backend:** Node.js with Express.js for API development
- **Database:** MongoDB for managing users, stories, chapters, and interaction data
- **AI Integration:** Google Gemini API for story generation and creative assistance
- **Storage:** Google Cloud Storage for media handling
- **Authentication:** JWT-based authentication system

2.4 Proposed Timeline (8 Weeks)

- Week 1: Requirements analysis and system design
 - Week 2: Backend development and authentication setup
 - Week 3: Frontend interface development
 - Week 4: AI integration for story generation
 - Week 5: Social feature implementation (likes, follows, story feed)
 - Week 6: Story thread feature and UI refinement
 - Week 7: Testing, debugging, and preparation for data collection
 - Week 8: Final evaluation, documentation, and presentation
-

3. Changes to the Proposal

3.1 Analytics System Simplified

Original Approach: The initial proposal included an automatic analytics logging system tracking detailed user activity metrics stored in MongoDB for quantitative analysis.

Change Implemented: Analytics tracking was simplified and partially deferred. The evaluation strategy now focuses primarily on qualitative feedback, structured surveys, and direct observation during testing sessions.

Justification: Building a full-scale analytics infrastructure would substantially increase backend complexity through additional schema planning, optimized aggregation queries, and performance tuning. Given the limited academic timeframe and small participant group (15-20 users), qualitative surveys and observed interaction patterns are sufficient to evaluate user motivation and engagement. Stability and feature completeness were prioritized.

3.2 Social Features Phased to Current Sprint

Original Approach: Likes, follows, story feeds, and the story thread feature were planned for completion in Weeks 5 and 6 of the proposal.

Change Implemented: Backend routes, models, and controllers for social features (like system, follow/unfollow, story model) were developed and committed to the repository. Full frontend integration is in progress for the current sprint.

Justification: AI integration (Google Gemini for outline and chapter generation) required more time than initially estimated due to prompt engineering challenges and API versioning updates (v0.x to v1.x breaking changes). Since AI-assisted writing is the core research feature, it was appropriately prioritized.

3.3 Authentication Approach Adjusted

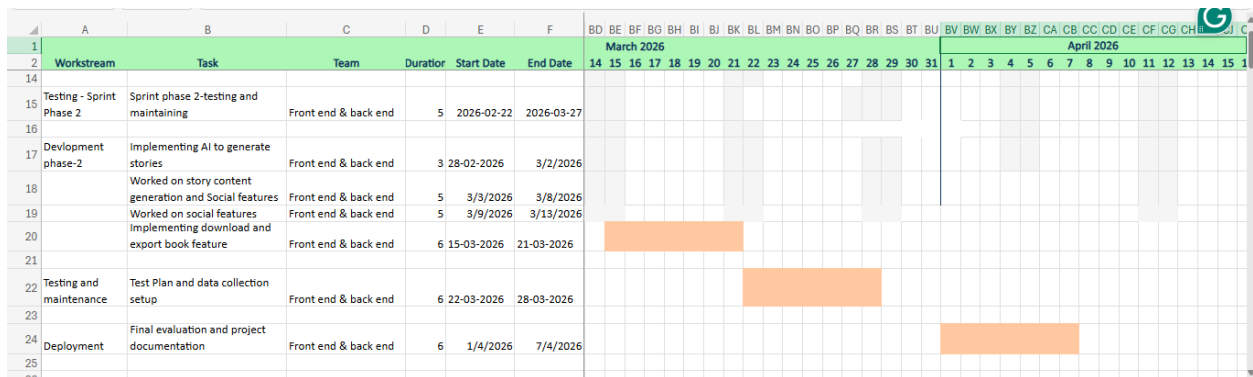
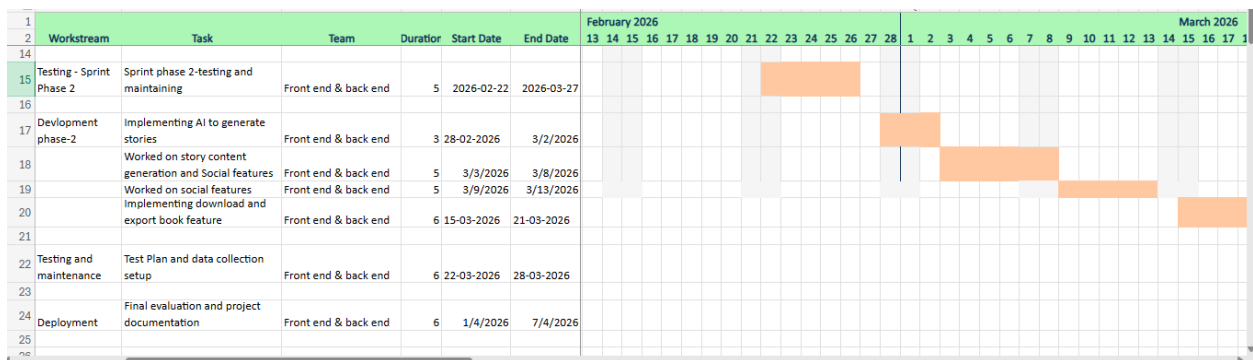
Original Approach: The proposal planned to use Firebase Authentication exclusively for user login and registration.

Change Implemented: JWT-based authentication with bcrypt password hashing was implemented using the MERN stack. Firebase Google OAuth is being integrated as a supplementary sign-in method in the current sprint.

Justification: JWT authentication was implemented first to establish a stable, controllable authentication foundation fully aligned with the MERN architecture. Firebase, while convenient for Google Sign-In, introduced additional dependency management and configuration complexity that would have delayed core backend completion.

4. Project Planning and Timeline (Current Period to End of Term)

- **22/02 – 28/02:** Integrating the frontend and backend for new features; testing and updating the system.
- **01/03 – 07/03:** Implementing the Google Gemini API to generate stories, descriptions, and content.
- **08/03 – 14/03:** Adding social features to the dashboard, such as "Like" and "Follow" functionality.
- **15/03 – 21/03:** Implementing the download feature to export books in PDF or Word format.
- **22/03 – 28/03:** Final testing and data collection setup.
- **01/04 – 07/04:** Final evaluation and project documentation.



5. Implemented Features

5.1 User Authentication System (Sign-Up and Sign-In)

Individual Contributions:

Aditi Aditi – backend (authController, authRoutes, authMiddleware, User model, bcrypt integration)

Khushi Bhatt – frontend (LoginPage, SignupPage, AuthContext, ProtectedRoute).

Feature Description

A secure user authentication system forms the access-control foundation of the Bookstagram platform. Since all core features including story creation, AI generation, and dashboard access are user-specific,

robust authentication was a foundational requirement from the start. The implemented system includes user registration, secure login, JWT-based session management, protected API routes, and password encryption using bcrypt.

How It Works

On the frontend, user credentials are collected using controlled form components managed with useState. When a user submits the form, an API request is sent to the backend using Axios.

On the backend:

- User data is validated before registration.
- Passwords are hashed using bcrypt before being stored in MongoDB.
- During login, the entered password is compared using bcrypt's comparison method.
- Upon successful authentication, a JWT token is generated and returned to the client.

Protected routes use middleware to verify the token before granting access to user-specific data such as profile information and dashboard content.

Challenges Faced

One challenge was ensuring secure password handling while still allowing proper validation during login. This required careful configuration of the Mongoose schema to prevent the password field from being returned unintentionally.

Another challenge involved correctly implementing JWT middleware for protected routes. Token validation errors initially caused unauthorized access issues, which were resolved through debugging token extraction and verification logic.

Screenshots – Login and Sign-Up Pages



Welcome Back

Continue your AI storytelling journey

Email

Password

[Forgot password?](#)

[Sign In to Bookstagram](#)

Don't have an account? [Join the community](#)

Figure 1: Login Page



Create an Account

Join the future of AI-powered storytelling.

Full Name

Email

Password

[Get Started](#)

Already have an account? [Sign in](#)

Figure 2: Sign-Up Page

5.2 Landing Page

Individual Contribution:

Khushi Bhatt and Aditi— all five sections (Navbar, Hero, Features, Testimonials, Footer) using React.js and Tailwind CSS.

All files committed by khushi

Feature Description

The landing page serves as the public entry point of the Bookstagram platform. Its primary purpose is to introduce the concept of AI-powered social publishing and guide users toward registration or login. The page is designed to communicate the platform's value proposition clearly while maintaining a modern and visually engaging interface.

The landing page consists of the following structured sections:

- Navigation Bar
- Hero Section
- Features Section
- Testimonials Section
- Footer

Each section is modularized into separate React components to improve maintainability and code organization

How It Works

The Navbar is fully responsive and dynamically adjusts based on authentication state using the `useAuth` context. If a user is logged in, a profile avatar and logout option are displayed; otherwise, Login and Get Started buttons are shown. The mobile menu toggling is managed with `useState` for a smooth responsive experience.

The Hero Section presents the core value proposition, displaying dynamic gradient typography and platform statistics (50K+ Books Created, 20K+ Stories Created, 4.9/5 Rating, 10min Average Creation Time). It includes a primary Call-to-Action button that conditionally navigates authenticated users directly to their dashboard.

The Features Section dynamically renders feature cards from a `FEATURES` data array. Each card includes an icon, title, description, and hover effects, highlighting AI-Powered Writing, Immersive Reader, One-Click Export, eBook Management, Build Your Fanbase, and Go Viral with Stories.

The Testimonials Section displays mapped user feedback cards, each containing star ratings, user image, quote text, and author details. This section enhances platform credibility through social proof.

The Footer contains structured navigation links, social media icons (Twitter, LinkedIn, GitHub), branding information, platform statistics (50K+ Happy Creators, 4.9/5 Average Rating, 100K+ eBooks Created), and a copyright notice.

Challenges Faced

One of the main challenges during landing page development was maintaining responsive consistency across different screen sizes. Ensuring proper alignment and spacing for large hero typography and grid layouts required multiple design refinements.

Another challenge involved implementing gradient effects and hover animations without affecting performance. Careful use of Tailwind utility classes was required to balance visual aesthetics with clean code structure.

Managing conditional rendering in the Navbar based on authentication state also required coordination with the AuthContext to prevent rendering inconsistencies.

Screenshots – Landing Page Sections

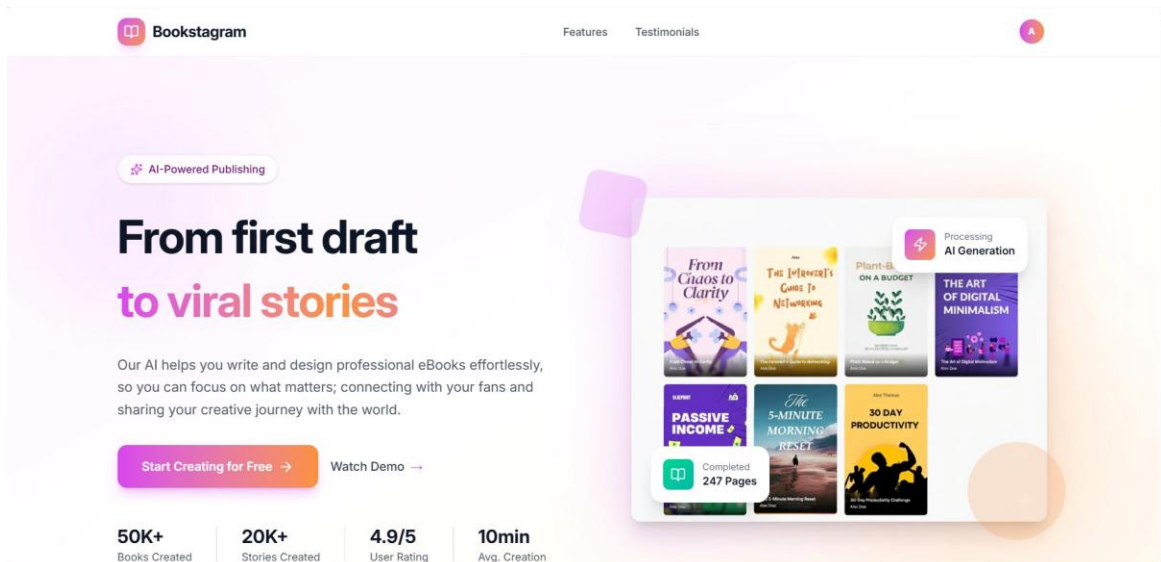


Figure 3: Landing Page-Hero Section

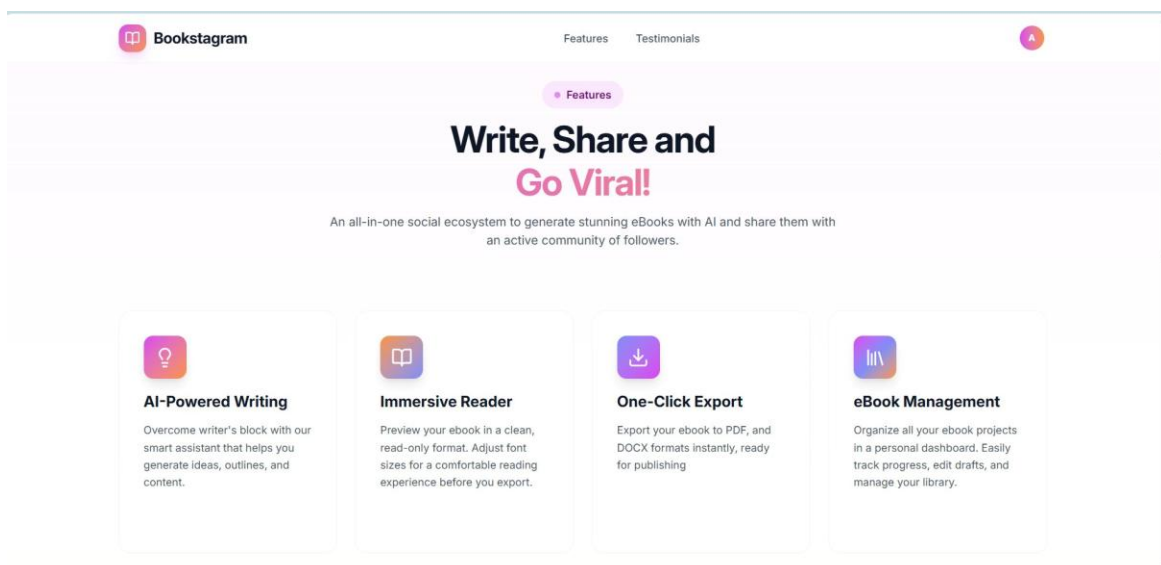


Figure 4: Landing Page – Features Section

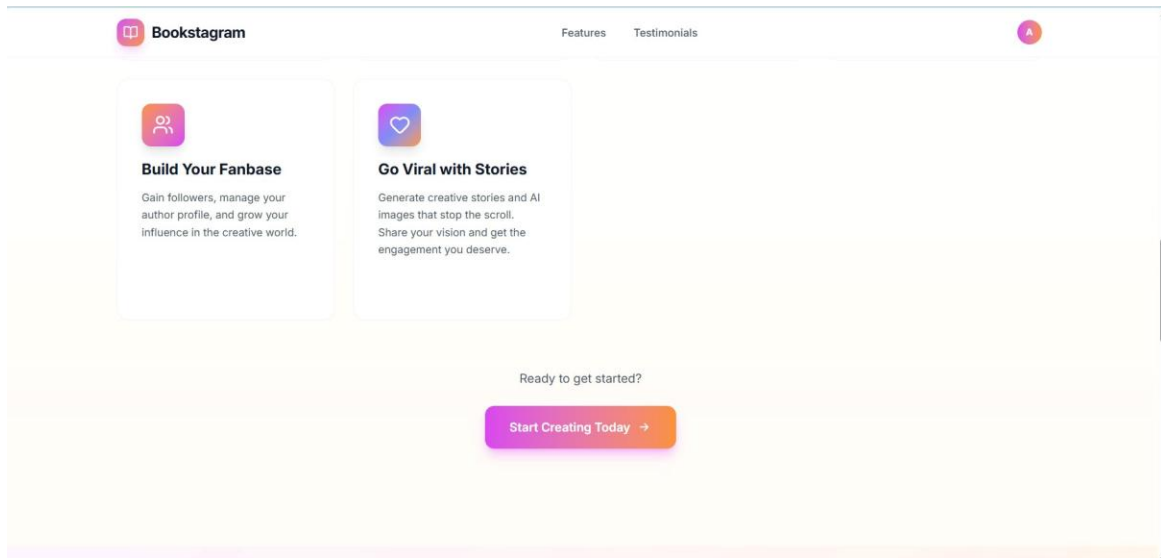


Figure 5: Landing Page

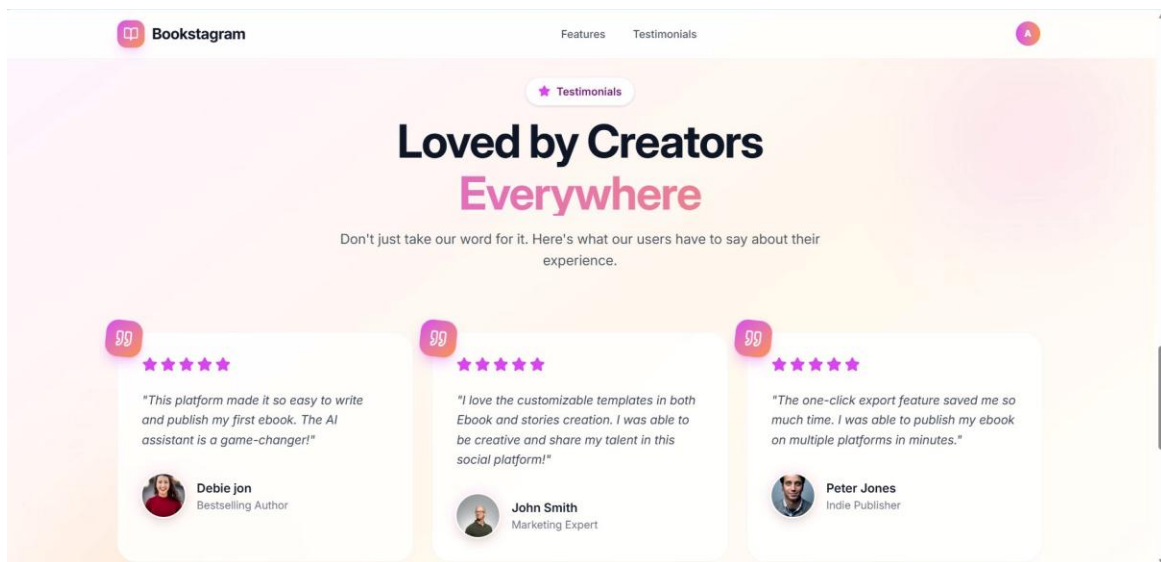


Figure 6: Landing Page – Testimonials Section

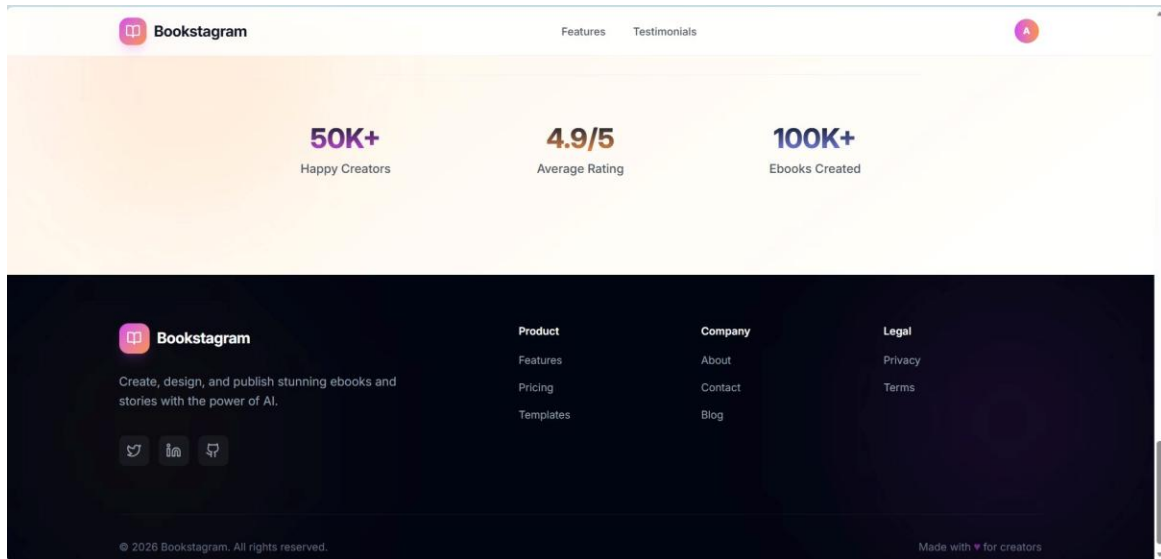


Figure 7: Landing Page – Footer

5.3 Dashboard and eBook Management System

Individual Contributions: Aditi Aditi – Dashboard layout, BookCard component, dashboard routes; Khushi Bhatt – CreateBookModal (two-step), delete confirmation modal, backend book API integration.

Feature Description

The Dashboard serves as the primary workspace for authenticated users. It allows users to create, view, and manage their AI-generated eBooks within a structured interface. Access to this page is restricted through authentication.

Key functionalities include:

- Viewing all created eBooks
- Creating new eBooks
- Deleting existing eBooks
- Redirecting to the editor page

How It Works

When the dashboard loads, a GET request is made to the backend to retrieve all eBooks associated with the logged-in user. The retrieved data is stored in React state and dynamically rendered using the reusable BookCard component, which displays the book cover image, title, and author name in a visually appealing card format. If no books exist, an empty-state UI with a call-to-action button is displayed.

Book creation is handled through a two-step CreateBookModal. Step 1 collects the Book Title, Number of Chapters, optional Topic, and Writing Style. The user clicks "Generate Outline with AI," which sends these parameters to the backend, where the Google Gemini API generates a structured chapter outline. Step 2 displays the generated chapters for review, with options to reorder or add chapters. Clicking "Create eBook" saves the book and chapters to MongoDB and redirects the user to the chapter editor.

Deletion is managed through a confirmation modal to prevent accidental removal. A DELETE request is sent to the backend, and the UI is immediately updated by filtering the local state without requiring a page refresh.

Challenges Faced

One challenge was ensuring real-time UI updates after book creation and deletion without requiring a page refresh. This required careful state management.

Another issue involved handling asynchronous API calls while maintaining smooth user experience through loading indicators and error notifications.

Screenshots – eBook Creation Modal and Dashboard

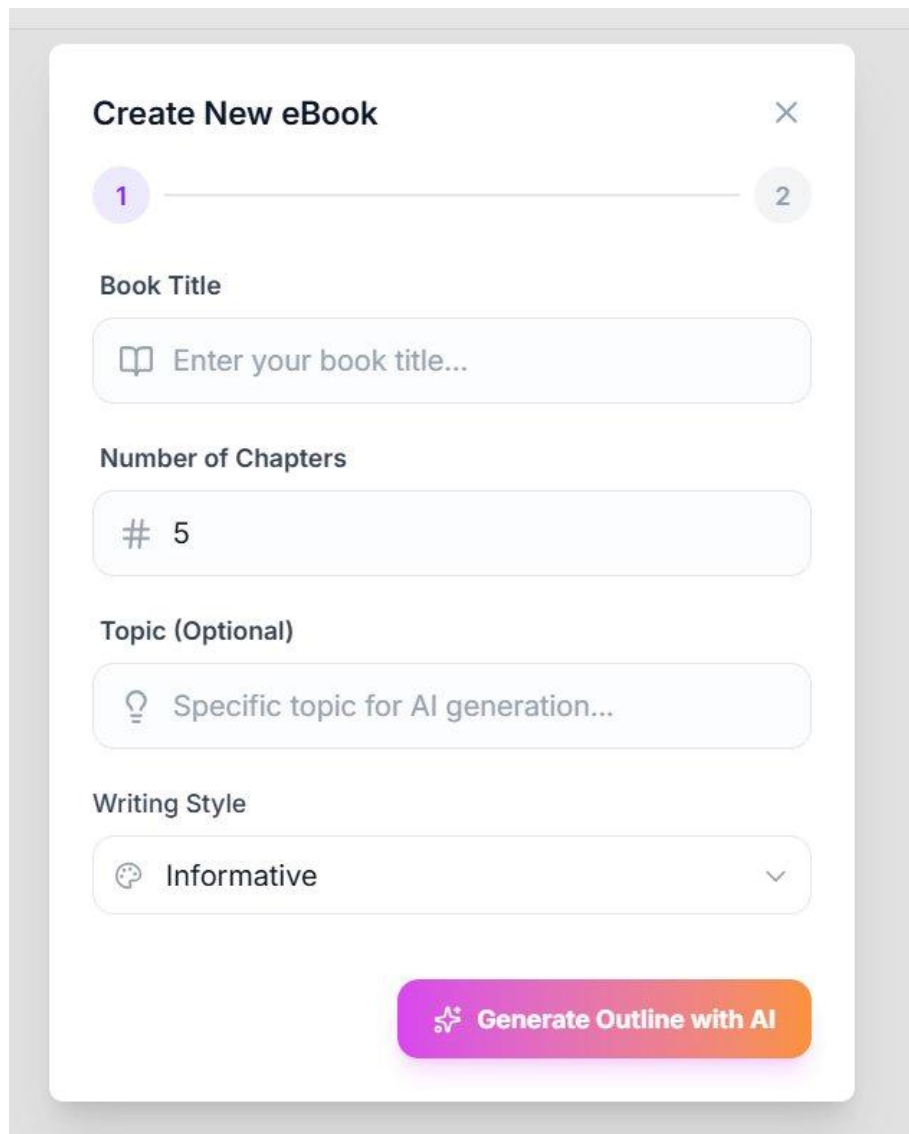
The image shows a 'Create New eBook' modal window. At the top, the title 'Create New eBook' is on the left and a close button 'X' is on the right. Below the title is a progress bar with two steps: '1' (active, highlighted in purple) and '2'. The form contains four input fields: 'Book Title' with a book icon and placeholder text 'Enter your book title...'; 'Number of Chapters' with a hash icon and the value '5'; 'Topic (Optional)' with a lightbulb icon and placeholder text 'Specific topic for AI generation...'; and 'Writing Style' with a palette icon, the value 'Informative', and a dropdown arrow. At the bottom right is a large, rounded button with a star icon and the text 'Generate Outline with AI'.

Figure 8: Create New eBook Modal

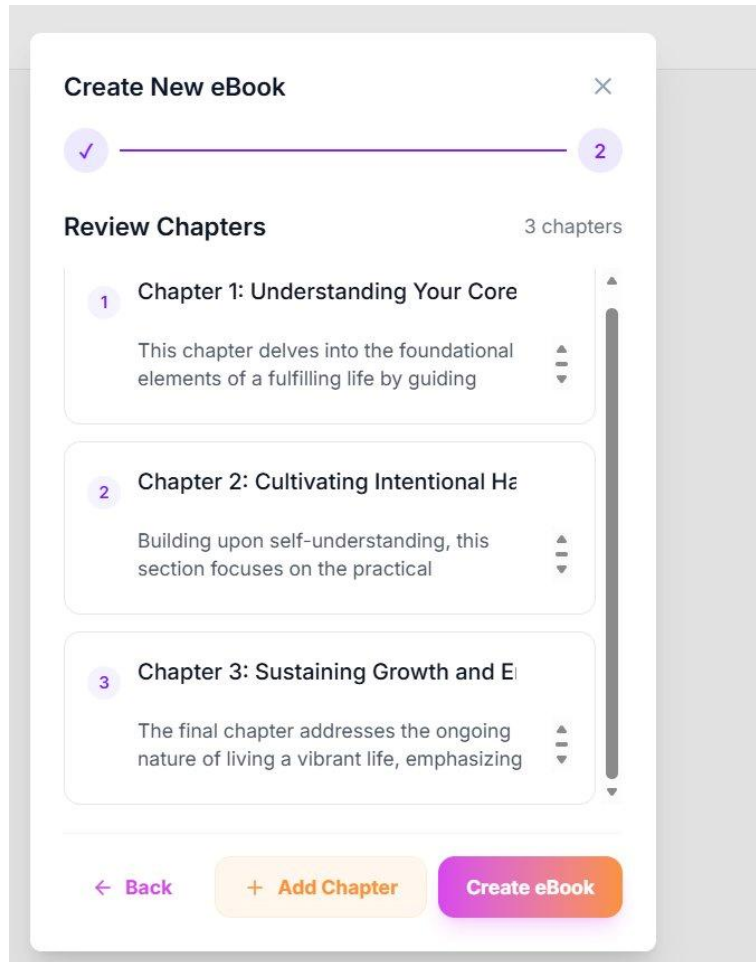


Figure 9: Create New eBook Modal

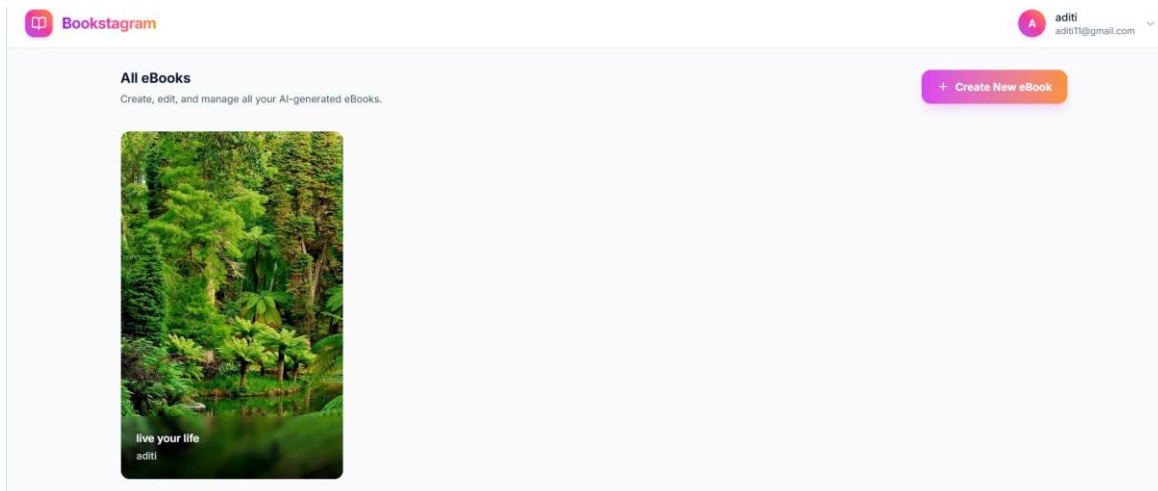


Figure 10: Dashboard

5.4 AI-Powered Chapter Editor with Markdown Support

Individual Contributions: Aditi Aditi – ChapterEditorTab, ChapterSidebar, SimpleMDEditor, BookDetailsTab components, ViewBook Component

Khushi Bhatt – Google Gemini AI content generation integration (aiController, aiRoutes), Save Changes API, chapter CRUD operations.

Feature Description

The Chapter Editor serves as the primary writing workspace of the Bookstagram platform. It integrates a split-pane Markdown editor with AI-assisted chapter content generation to support structured and interactive writing. The interface is divided into two main tabs: the **Editor** tab, which allows users to write and generate chapter content, and the **Book Details** tab, which enables users to manage metadata such as the book title, author name, subtitle, and cover image.

A dedicated sidebar on the left displays all chapters associated with the current eBook, providing efficient navigation and allowing users to switch between chapters seamlessly.

How It Works

When a user navigates to the Editor Page from the dashboard, the system retrieves the selected eBook and its chapters from the backend database. The **ChapterSidebar** component displays all chapter titles as interactive list items, with the active chapter highlighted for clarity. Users can also add new chapters directly through the sidebar interface.

The central content area renders the **ChapterEditorTab**, which includes an editable chapter title field and a Markdown-based editor (SimpleMDEditor). The editor operates in a split-pane mode by default, displaying raw Markdown input on one side and a live formatted preview on the other. The toolbar provides multiple formatting tools, including bold, italic, headings, lists, links, code blocks, images, and full-screen editing functionality.

The **“Generate with AI”** feature enables users to request AI-generated chapter content. Upon activation, the chapter title and relevant book context are sent to the backend AI controller. The controller constructs a structured prompt and communicates with the Google Gemini API. The generated content is returned as Markdown text and inserted directly into the editor. Users retain full control and may modify the generated content before saving.

The **Preview mode** renders the Markdown content as formatted HTML, allowing users to review how the chapter will appear once published.

The **Book Details** tab permits updates to book metadata and supports cover image uploads. Uploaded images are processed through the backend and stored securely, after which they are displayed both within the editor and on the dashboard book card.

The top navigation bar includes an **Export** dropdown (supporting PDF and DOCX formats) and a **Save Changes** button, which persists all edits to the MongoDB database.

Challenges Faced

Integrating the Google Gemini API required careful prompt engineering to ensure consistently structured and coherent chapter output. A significant technical challenge arose due to breaking changes between @google/genai API versions (v0.x to v1.x), which required updating the model configuration and request format.

Screenshots – Chapter Editor Workflow

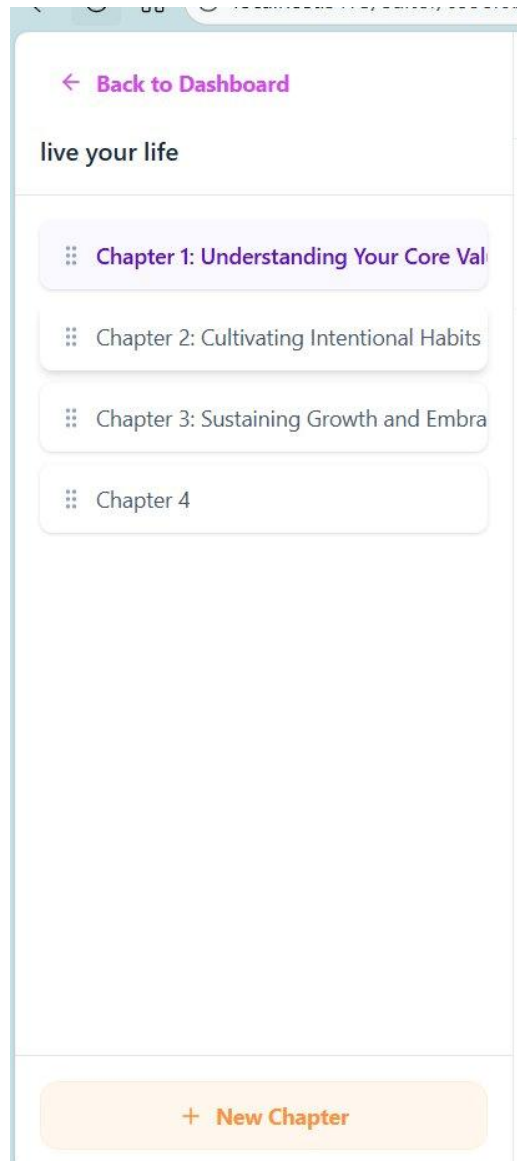


Figure 11: Chapter Sidebar

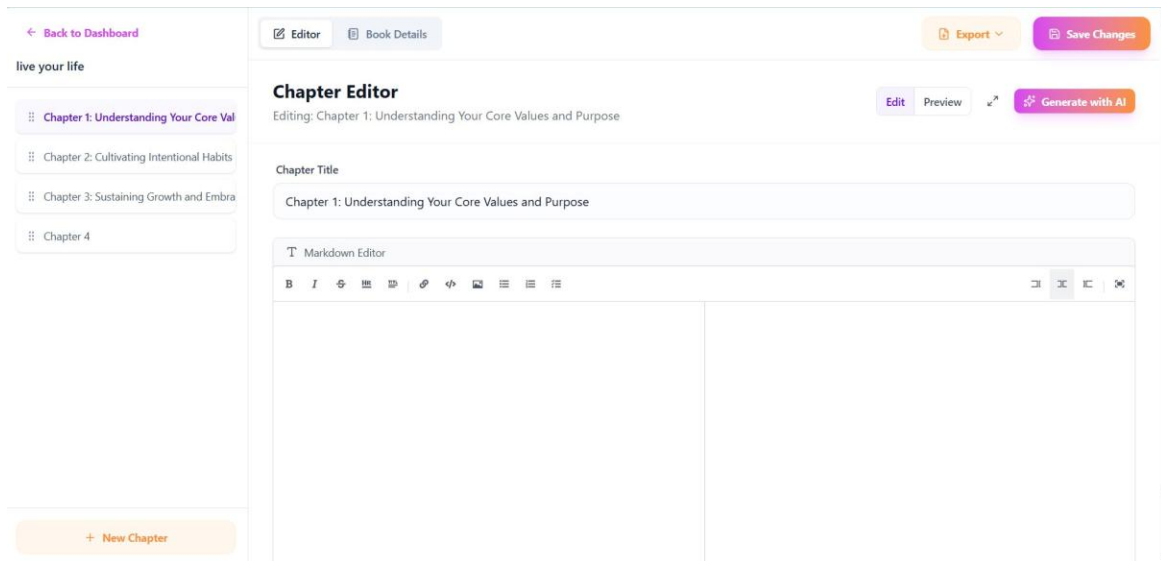


Figure 12: Chapter Editor

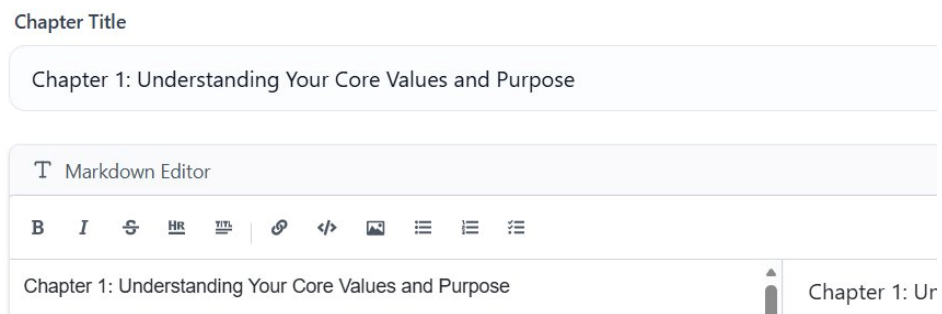


Figure 13: Markdown Editor Toolbar

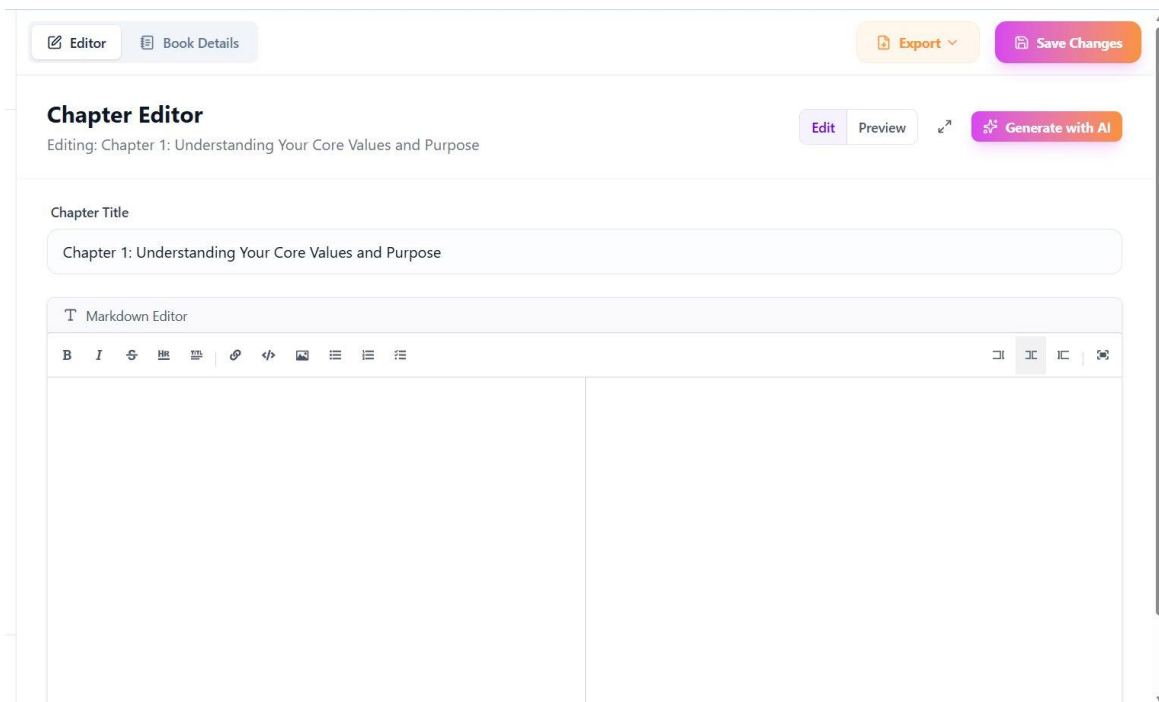


Figure 14: Chapter Editor

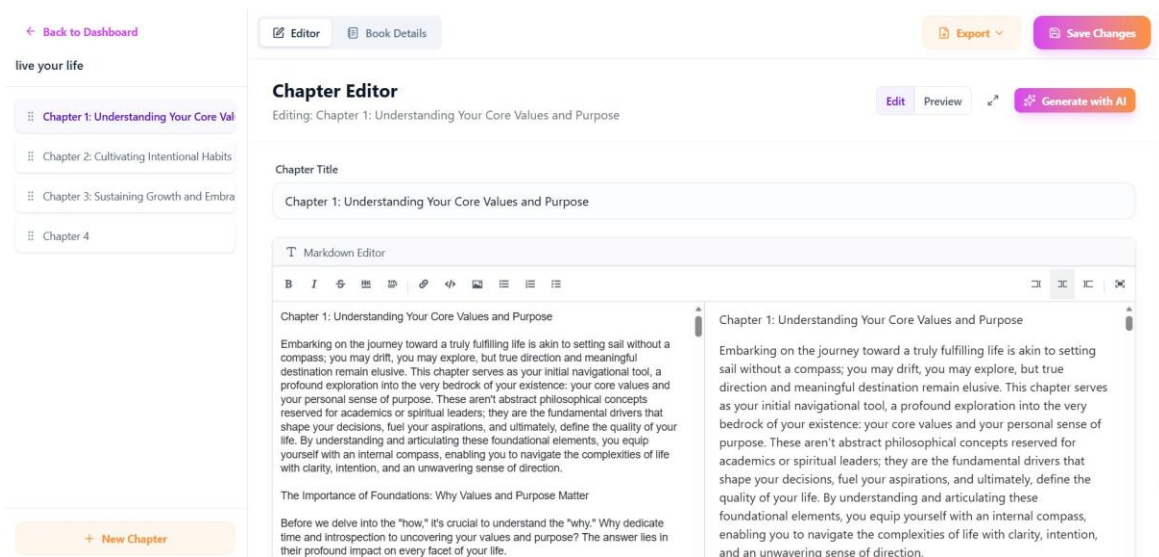


Figure 15: Chapter Editor

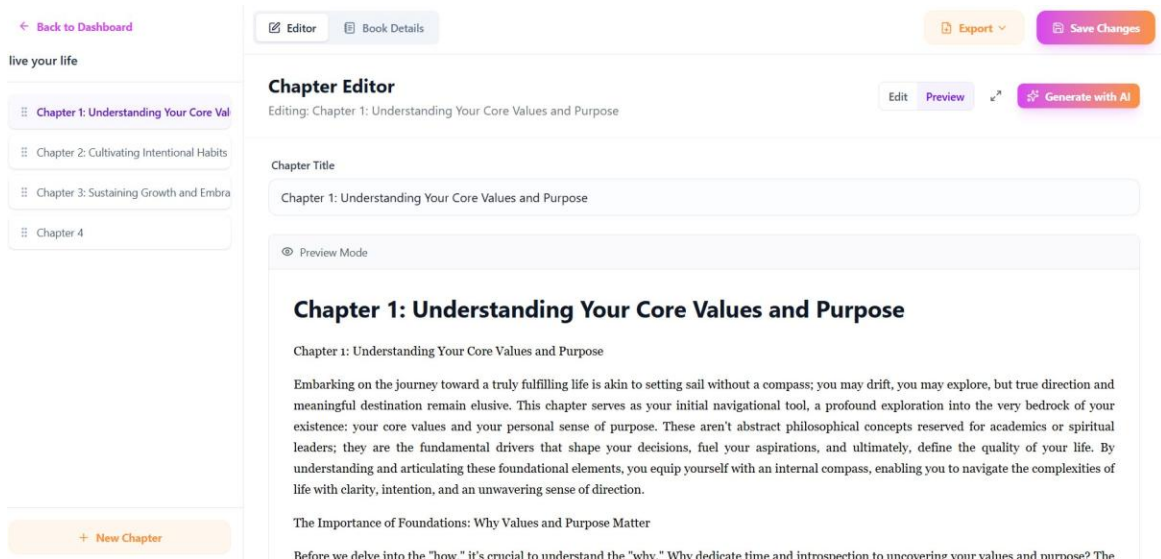


Figure 16: Chapter Editor

5.5 Book Details Tab and Cover Image Upload

Individual Contributions: Aditi Aditi – BookDetailsTab UI component,

Khushi Bhatt – cover image upload API (multer middleware), backend storage, and Book model update.

Feature Description

The Book Details tab within the chapter editor provides users with a structured interface to manage the eBook's metadata and visual identity. Users can update the book title, author name, and subtitle, and upload a custom cover image for their eBook. The uploaded cover image is stored on the server and displayed on both the Book Details tab and the Dashboard book card.

How It Works

When a user navigates to the Book Details tab, the current book metadata is loaded from state. Users can edit the Title, Author, and Subtitle text fields. The cover image section displays the current cover (or a placeholder if none exists) alongside an "Upload Image" button. Clicking this button opens a file picker; the selected image is sent to the backend via a multipart form POST request handled by the multer middleware. The backend saves the file to the uploads folder and returns the image URL, which is saved to the Book document in MongoDB. The cover image immediately updates in the UI upon successful upload.

Challenges Faced

Configuring the multer middleware for file uploads required proper setup to manage file size limits, validate image types, and define secure storage paths. It was important to ensure that only appropriate image files could be uploaded.

Another challenge was making sure that the uploaded cover image URL was correctly displayed in both the editor page and the dashboard book card. This required consistent URL handling and proper state synchronization between frontend components to ensure the image updated immediately after upload.

Screenshots – Book Details Tab

The screenshot shows the 'Book Details' tab in a web application. At the top, there are two tabs: 'Editor' and 'Book Details', with 'Book Details' being the active tab. To the right of the tabs are two buttons: 'Export' (orange) and 'Save Changes' (purple). The main content area is divided into two sections. The first section, titled 'Book Details', contains three input fields: 'Title' with the text 'live your life', 'Author' with the text 'aditi', and 'Subtitle' which is empty. The second section, titled 'Cover Image', features a placeholder image labeled 'Cover' and a text prompt: 'Upload a new cover image. Recommended size: 600x800px.' Below the prompt is an orange button labeled 'Upload Image'.

Figure 17: Book Details Tab

This screenshot shows the same 'Book Details' tab as Figure 17, but with a different cover image. The 'Title' field still contains 'live your life' and the 'Author' field contains 'aditi'. The 'Subtitle' field remains empty. In the 'Cover Image' section, the placeholder image has been replaced by a photograph of a dense green forest. The text prompt 'Upload a new cover image. Recommended size: 600x800px.' and the 'Upload Image' button are still present.

Figure 18: Book Details Tab

5.6 Profile Tab

Feature Description

The Profile Page allows users to manage their personal account information. It provides a simple interface where users can view and update their full name and email address. This feature ensures that user account details remain accurate and up to date.

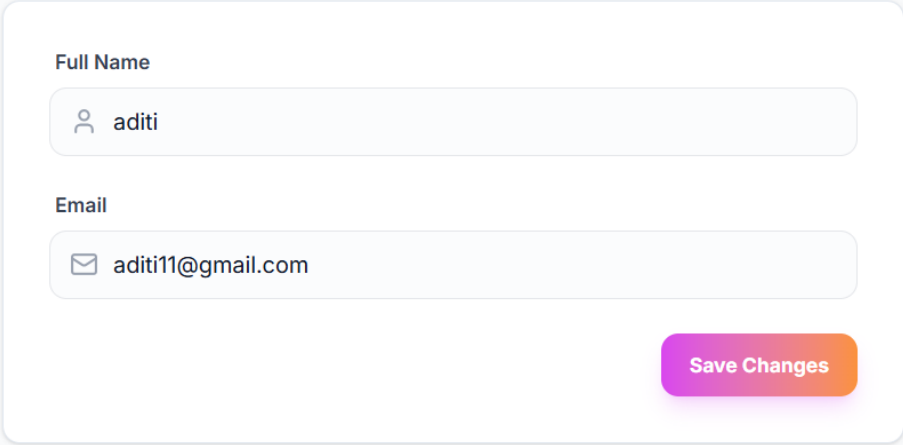
The page includes editable input fields and a “Save Changes” button to confirm updates.

How It Works

When the user navigates to the Profile page, the system retrieves their account information from the backend using a protected API route. The full name and email address are displayed in editable input fields.

If the user updates their name and clicks “**Save Changes**”, a PUT request is sent to the backend to update the user record in the MongoDB database. After successful completion, a confirmation message is displayed to indicate that the profile has been updated.

Authentication is handled using JWT tokens to ensure that only the logged-in user can access and modify their own profile information.



The image shows a web form titled "Profile" with the subtitle "Manage your account details." The form contains two input fields: "Full Name" with the value "aditi" and "Email" with the value "aditi11@gmail.com". A "Save Changes" button is located at the bottom right of the form.

Profile
Manage your account details.

Full Name
aditi

Email
aditi11@gmail.com

Save Changes

Figure 19: User Profile

6. AI Use Section [Combining prompt of Khushi's and Aditi's AI Usage]

AI Tool Name	Version / Account Type	Specific Feature for which AI Tool was Used	Value Addition (What value did you add beyond the AI?)
ChatGPT (OpenAI)	GPT-4, Free Version	Landing Page UI Development; structure and suggestions for Navbar, Footer, Features section, Testimonials, Buttons, and Input Fields	Structured the Frontend folder structure with different reusable components for the landing page. Worked on consistent styling like buttons, hero image, border and writing style.
ChatGPT (OpenAI)	GPT-4, Free Version	Understanding authentication flow: controllers, routes, and middleware integration.	Manually built and tested the complete auth flow end-to-end. Resolved token validation bugs through independent debugging and code review.
ChatGPT (OpenAI)	GPT-4, Free Version	Story Generation Backend Logic; creation of Story Model, Story Routes, Story Controller, and debugging 500 error in book outline generation.	Created MongoDB schema for stories according to project requirement. Modified AI-generated structure for the new feature on the backend structure. Debugged the 500 error by generating gemini API key configuration. Implemented try-catch for error handling.
ChatGPT (OpenAI)	GPT-4, Free Version	Outline Generation Error Handling; Debugged backend issues related to AI-based outline generation flow.	Analyzed server logs and identified the exact failure point causing the 500 error instead of relying only on AI suggestions. Validated environment variables and corrected API configuration issues.
ChatGPT (OpenAI)	GPT-4, Free Version	Assistance in resolving implementation errors and understanding server-side error messages.	Read and interpreted server error messages instead of just copying fixes directly. Matched error line numbers with actual code to identify the issue. Checked middleware, routes, and controller connections.

			Tried small code changes and re-ran the server to check what was causing the problem.
ChatGPT (OpenAI)	GPT-4, Free Version	Branch Merging & Conflict Resolution Support, and Ebook creation functionality.	Manually reviewed conflicting files during branch merge instead of blindly accepting changes. Compared both versions of the code and kept the correct logic based on latest implementation. Tested the project after merging. Made minor adjustments in controller/routes to ensure ebook creation flow worked
ChatGPT (OpenAI)	GPT-4, Free Version	Updated model configuration for @google/genai v1.x, reviewed API endpoints, updated authentication routing in App.jsx, and supported end-to-end feature testing.	Updated model name and request format to match the new @google/genai v1.x version. Reviewed API endpoints manually to ensure routes, controllers, and responses were properly connected. Fixed small mismatches between frontend requests and backend responses. Perform testing for the whole project after modification.
Free Grammar Checker	Free Version	Grammar and spelling review of report sections and documentation.	All content is written independently. Grammar tool used only for final proofreading; all corrections manually reviewed and accepted.

7. Work Logs

7.1 Khushi Bhatt (Team Lead) – 300394398 [After report 1]

Date	Hours	Description of Work Done
9/2/2026	4	Worked on the report. Worked on Navbar, footer, features, input fields, testimonials, buttons and completed the landing page.
10/2/2026	3	Worked on landing page components, applied some changes in the UI.
11/2/2026	4	Change some content on the landing page & change the UI theme of the sign-up and login pages. Merge 2 branches.
12/2/2026	3	Worked on source control. TypeError: argument handler must be a function
13/02/2026	3	Worked on AI Controller, AI Routes in the backend. Solved error in the routes and gemini API key.
14/02/2026	4	Created story model, story routes and story Controller. Fixed 500 error in the book outline generation.
15/02/2026	2	Modified Bookroutes and Book Controller.
18/02/2026	3	Working on the backend error occurring from outline generation.
19/02/2026	3	Removed nodemodules from backend and frontend. Relocate .gitignore file to implementation folder. Modified Profile page.
20/02/2026	4	Merged 2 branches for the sprint phase 2, solved conflicts of source control. Testing outline generation and ebook creation. Working on the midterm report collaboratively with Aditi.
21/02/2026	2	Modified bookroutes and server.js. Continue worked on midterm report.

7.2 Aditi, Aditi (Team Member) – 300396361

Date	Hours	Description of Work Done
Feb 8, 2026	1.5	Modified authentication controller and routes; improved middleware structure. Commits: authRoute, auth middleware, authController
Feb 9, 2026	2	Diagnosed and resolved server-side errors related to backend configuration and API handling. Conducted debugging and tested system stability.
Feb 12, 2026	3	Implemented dashboard route; created BookCard component; structured dashboard UI and debugged rendering errors. Merged pull request #4.
Feb 13, 2026	5	During this phase, I worked on building the main Editor page of the project. I created components like the chapter sidebar, chapter editor tab, and book details tab to make the writing process organized and user-friendly. Add some frontend files such as SimpleMDEditor, ViewBook and ViewChapterSidebar.
Feb 14, 2026	1.5	Worked on backend files collaboratively with Khushi; reviewed and tested API endpoints.
Feb 16, 2026	1	Diagnosed and resolved 500 server error in generate-outline endpoint related to Gemini API configuration.
Feb 17, 2026	2.5	Worked on ViewBook page in frontend; solved errors in AiController related to model name changes in @google/genai v1.x.
Feb 18, 2026	1	Started working on midterm report; organized documentation and gathered feature details.
Feb 19, 2026	2	Worked on App.jsx routing; changed authentication system configuration; modified ProfilePage per project requirements; added new API endpoints.
Feb 20, 2026	3	Working on midterm report collaboratively with Khushi; finalized feature documentation and screenshots.
Feb 21, 2026	2	Recorded video for midterm report demo; tested all implemented features end-to-end.

8. Closing and References

Acknowledgements

The team would like to express sincere gratitude to Professor Padmapriya Arasanipalai Kandhadai for her continuous guidance, constructive feedback, and support throughout the research and development process. We also thank the early testers who provided valuable feedback on the platform prototype. The development of Bookstagram was supported by the open-source communities surrounding MongoDB, Express.js, React.js, Node.js, and the Google Gemini AI API.

References

- Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? Proceedings of the ACM Conference on Fairness, Accountability, and Transparency, 610-623.
- Google. (2024). Google Gemini API Documentation. <https://ai.google.dev/gemini-api/docs>
- MongoDB. (2024). MongoDB Documentation. <https://www.mongodb.com/docs/>
- React. (2024). React Documentation. <https://react.dev/>
- Tailwind CSS. (2024). Tailwind CSS Documentation. <https://tailwindcss.com/docs>
- JWT.io. (2024). Introduction to JSON Web Tokens. <https://jwt.io/introduction>
- Vistasocial. (2024). Generative AI in content creation. <https://vistasocial.com/insights/generative-ai/>
- Combinedbook. (2024). How AI is changing the landscape of book publishing. <https://www.combinedbook.com/how-ai-is-changing-the-landscape-of-book-publishing-and-copyright/>